

PROJECT REPORT

Space Shooter

A Python / Pygame Wave-Based Arcade Game

Group Members

Name	Student ID
Shadman Abrar	22-47687-2
Songlap Kumar Shuttadhar	22-47633-2
Sheikh Syeed Ul Haque	22-48264-2

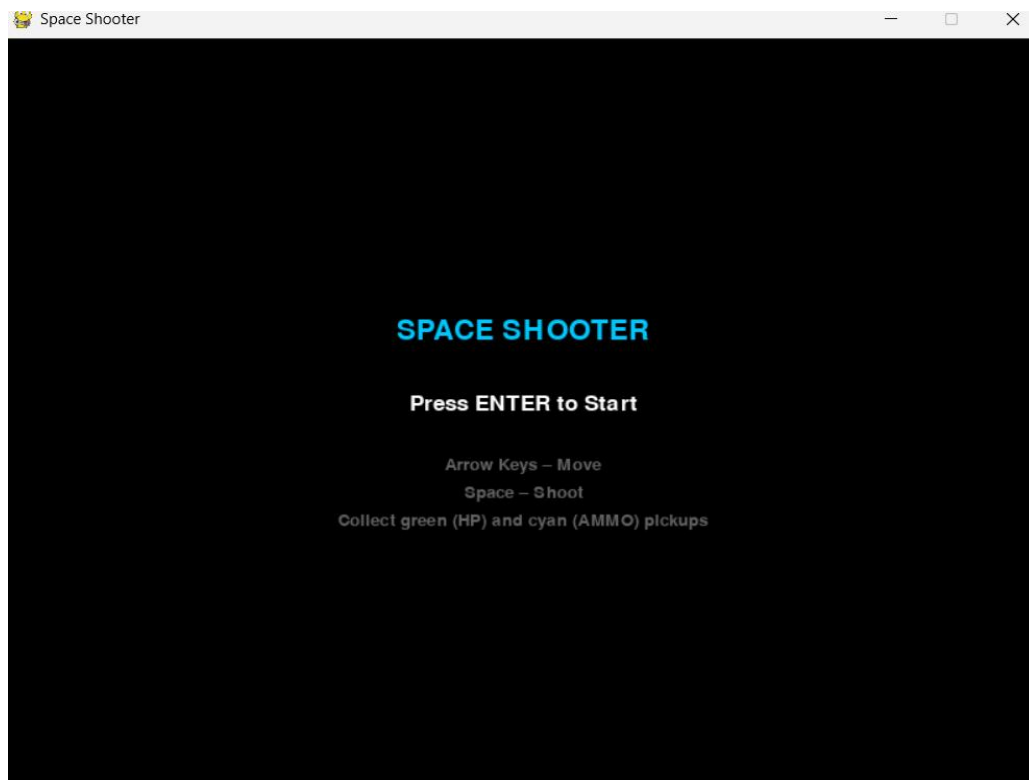


Fig 1 — Title / Start Screen

Overview

Space Shooter is a top-down, wave-based arcade game built entirely in Python using the Pygame library. The player controls a spaceship navigating a scrolling star field, shooting down waves of increasingly difficult enemies. Every fifth wave replaces regular enemies with a multi-phase boss fight. Between waves, defeated enemies drop health and ammo pickups the player must collect to survive. A JSON-based save system tracks the player's all-time high score, best wave, total enemies killed, and total games played across sessions.

Project Name	Space Shooter
Track / Genre	Arcade / Wave Shooter
Language	Python 3.13
Framework	Pygame 2.6.1
Window Size	800 × 600 px 60 FPS
Save File	data.json (local, auto updated on death)
Entry Point	main.py

Setup & Installation

Prerequisites: Python 3.10+ and pip installed on your system.

Step 1 — Clone or extract the project

```
unzip Project_U.zip
cd Project
```

Step 2 — Install the only required runtime dependency

```
pip install pygame==2.6.1
```

The requirements.txt in the zip is a full environment dump and contains many unrelated packages. Only pygame is required to run the game.

Step 3 — Run the game

```
python main.py
```

No build step, no configuration file, no database — the game launches immediately. A data.json save file is created automatically on first run.

How to Play

Arrow Keys	Move the ship in all four directions
Space	Fire bullets (250 ms cooldown)
Enter	Start game from the title screen
R	Restart after Game Over
Escape	Quit at any time

The objective is to survive as many waves as possible and achieve the highest score. Shoot enemies to earn points, collect pickups to replenish HP and ammo, and defeat the boss that appears every fifth wave.

Feature List

Core Gameplay

- Wave-based progression — enemy count, speed, and health all scale with each wave number
- Enemy color changes per wave through red, orange, yellow, cyan, and purple
- Enemies enter from above with staggered spawning (one enemy released every 40 frames)
- Contact damage — any enemy that reaches the player deals 20 HP on collision
- Ammo is finite (max 200); the player cannot shoot if ammo hits zero

Two-Phase Boss Fight

- Boss spawns every 5th wave and replaces the normal enemy formation
- Phase 1 — purple hexagonal ship, slow movement, single-bullet shots every 90 frames
- Phase 2 — triggers at 50 % HP; color shifts to orange, movement doubles in speed, fires a 3-bullet spread every 45 frames
- Boss HP increases by 100 per additional boss cycle (wave 10 boss has 400 HP, wave 15 has 500, etc.)
- Boss displays a live HP bar and a "!! PHASE 2 !!" label during the enraged state

Pickups

- 30 % chance to drop a pickup when any enemy is killed
- Green pickup — restores 15 HP (capped at 100)
- Cyan pickup — restores 15 ammo (capped at 200)
- Pickups drift downward and despawn off-screen if not collected

HUD & Feedback

- Top-left health bar — color shifts green → yellow → red as HP drops
- Ammo bar below health — shifts cyan → orange below 25 %
- Top-right score and all-time best score display
- Wave number shown in cyan top-right

- Boss flash-white effect on every hit for clear damage feedback

Persistence & Save System

- SaveManager writes to data.json on every game-over automatically
- Tracks: high_score, best_wave, total_games, total_enemies across all sessions
- Title screen shows all-time best score and best wave if any game has been played
- "★ NEW RECORD! ★" message flashes on the game-over screen when a new high score is set

Visual Effects

- Animated parallax star field — 80 stars with varied speeds and brightness
- All game art is drawn procedurally with Pygame primitives (no external sprite sheets)
- Three game states: START → PLAYING → GAME_OVER with clean transitions

Screenshots

Gameplay — Fighting Enemies

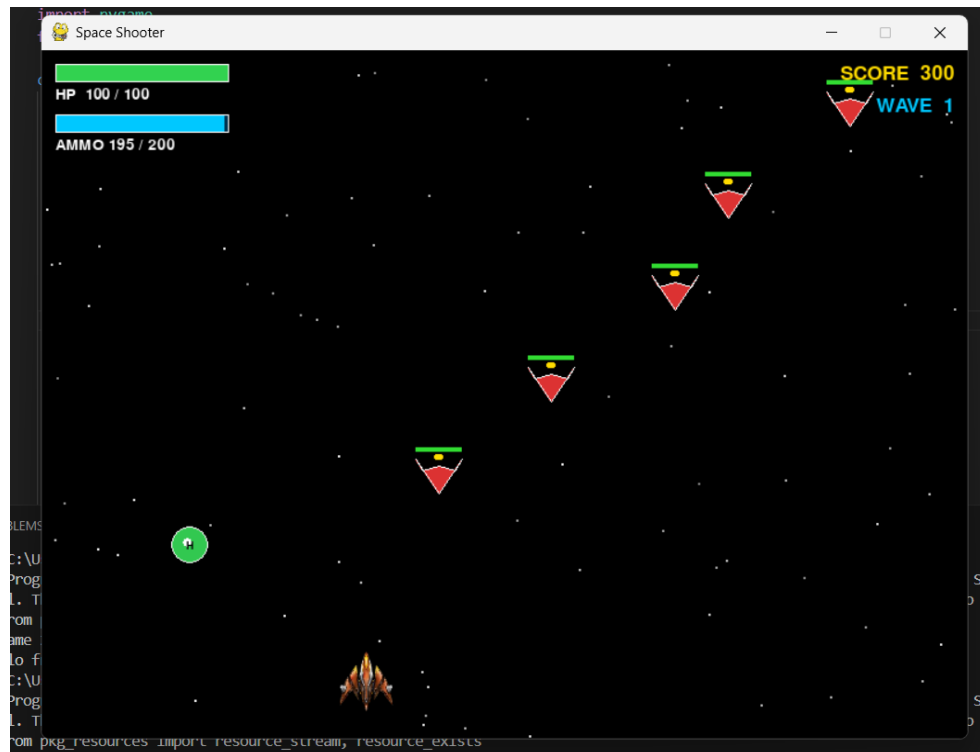


Fig 2 — Wave in progress: enemies descending, HUD visible

Boss Fight — Phase 1

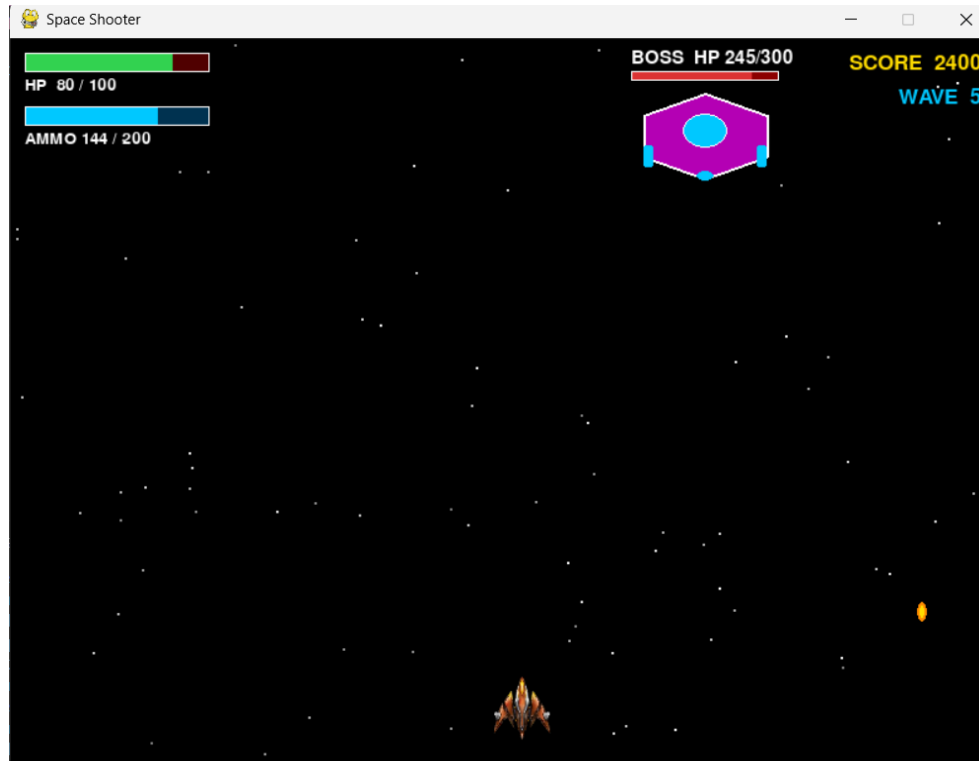


Fig 3 — Boss Phase 1 (purple): single-bullet attack pattern

Boss Fight — Phase 2

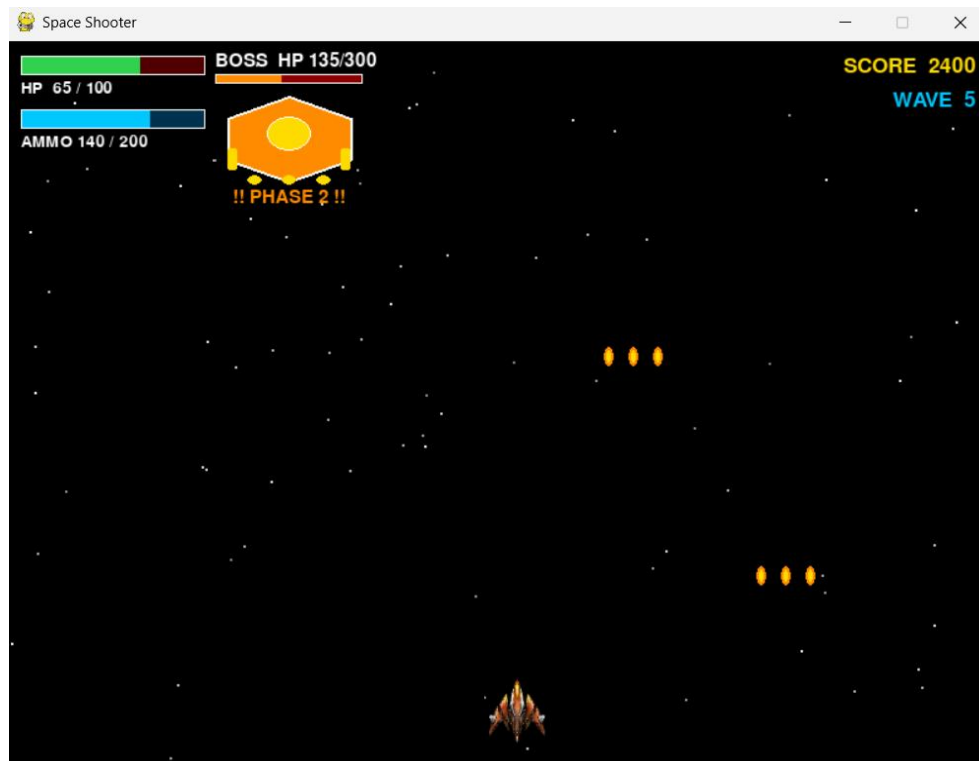


Fig 4 — Boss Phase 2 (orange): speed doubled, triple-spread bullets, "!! PHASE 2 !!" label

All Time High-Score and Wave

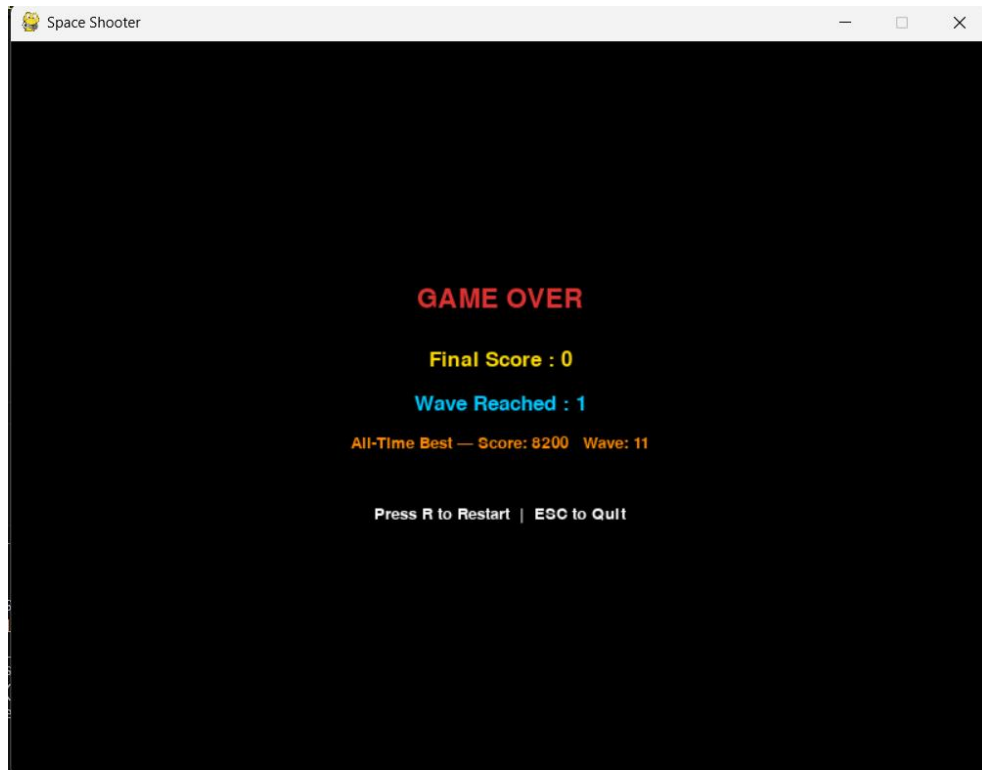


Fig 5 — Best score and Wave number

Player Ship



Fig 6 — Player ship

Project Structure

main.py	Game loop, state machine (START / PLAYING / GAME_OVER), input handling, collision resolution
settings.py	All tunable constants: window size, FPS, speeds, damage, colours
player.py	Player ship — movement, shooting, health/ammo management
enemy.py	Standard enemy — movement, hit detection, damage
boss.py	Boss + BossBullet — two-phase AI, firing patterns, HP bar render
wave_manger.py	WaveManager — wave sequencing, enemy/boss spawning, bullet–entity collision, pickup drops, scoring
Hud.py	All on-screen UI: HP bar, ammo bar, score, wave, start/game-over screens
bullet.py	Player bullet — upward movement, collision flag
pickup.py	Pickup item — downward drift, apply effect on collection
save_manager.py	SaveManager — read/write data.json, update records after each game
data.json	Persistent save data (auto created / updated at runtime)

Tunable Constants (settings.py)

PLAYER_SPEED	5 px/frame
MAX_HEALTH	100
MAX_AMMO	200
BULLET_SPEED	35 px/frame
BULLET_DAMAGE	55
SHOOT_COOLDOWN	250 ms
ENEMY_SPEED_BASE	2 px/frame (scales +0.3 per wave, capped at 6)
ENEMY_HEALTH	40 HP (scales +10 per wave)
WAVE_SIZE	8 enemies (scales +2 per wave, capped at 20)
BOSS_HEALTH	300 HP base (+100 per boss cycle)
HEALTH_PICKUP_VALUE	15 HP
AMMO_PICKUP_VALUE	15 ammo
PICKUP_DROP_CHANCE	30 %
Points per enemy	100
Points per boss	1000